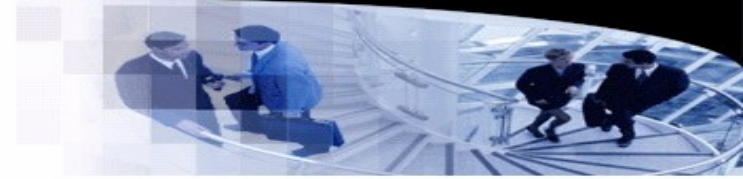




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

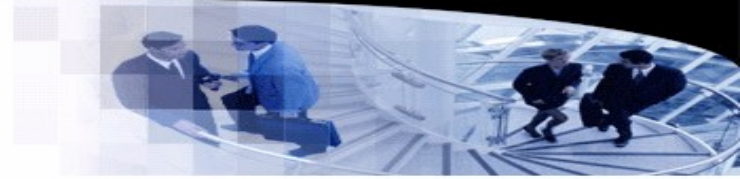
Lecture 4. Formal Languages

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

Alphabet, Strings, and Languages



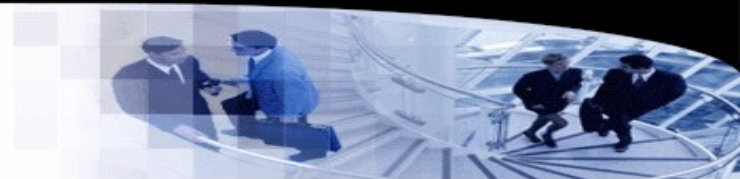
- Formal languages

- Languages whose **syntax** and **semantics** are rigorously defined.
- A formal languages consists of strings whose letters are taken from an **alphabet** according to a formal **grammar**.

- Alphabet

- A non-empty finite set.
- Elements in an alphabet are called **symbols**.
- Examples:
 - $\{0, 1\}$, $\{a, b, \dots z\}$, $\{0x00, 0x01, \dots 0x7f\}$

Alphabet, Strings, and Languages



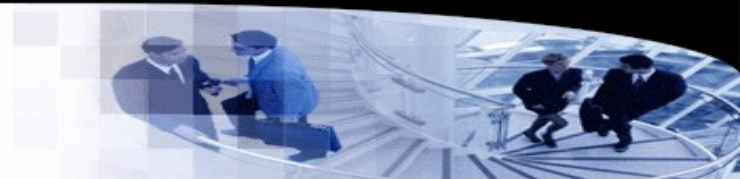
- **Strings**

- A string is a sequence of symbols.
- How to define strings with mathematical notations?
- ϵ denotes the empty string that contains no symbols.
- The length of a string α is the number of symbols in the string, denoted as $|\alpha|$.

- **Concatenation**

- The concatenation of two strings α and β is denoted as $\alpha \cdot \beta$ or $\alpha\beta$.
- $\epsilon \cdot \alpha = \alpha \cdot \epsilon = \alpha$
- $\alpha^0 = \epsilon$
- $\alpha^1 = \alpha, \alpha^2 = \alpha\alpha, \dots, \alpha^n = \alpha\alpha^{n-1}$
- Write an OCaml program which calculates the concatenation of two strings.

Alphabet, Strings, and Languages



- **Multiplication**

- Let A and B be two sets of strings. The multiplication of A and B is denoted as $A \cdot B = AB = \{\alpha\beta \mid \alpha \in A, \beta \in B\}$.
- $\{\epsilon\} \cdot A = A \cdot \{\epsilon\} = A$
- $\emptyset \cdot A = A \cdot \emptyset = \emptyset$
- $A^0 = \{\epsilon\}$
- $A^1 = A, A^2 = AA, A^n = AA^{n-1}$

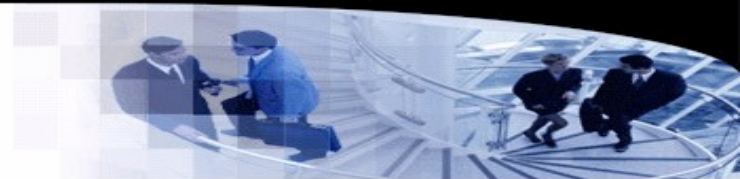
- **Kleene star (or Kleene closure or Kleene operator)**

- $A^* = A^0 \cup A^1 \cup A^2 \cup \dots$

- **Kleene plus**

- $A^+ = A \cdot A^* = A^1 \cup A^2 \cup A^3 \cup \dots$

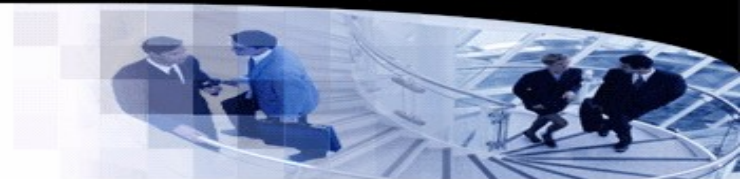
Alphabet, Strings, and Languages



- Examples

- $\{a, b\} \cdot \{c, d\} = \{ac, ad, bc, bd\}$
- $\{a, aa\} \cdot \{ab, b\} = \{aab, ab, aaab\}$
- Let $\Sigma = \{0, 1\}$.
- $\Sigma^2 = \{00, 01, 10, 11\}$ is the set of all 2-bit binary numbers.
- What about Σ^3 and Σ^4 ?
- $\Sigma^* = \Sigma \cup \Sigma^2 \cup \Sigma^3 \cup \dots$. What is the meaning of Σ^* ?

Alphabet, Strings, and Languages



- Language

- A language is a **set** of strings from a given alphabet.

- Defining a language

- By defining a set:

- $\{1, 2, 3\}$
- $\{0^n 1^n \mid n > 0\}$

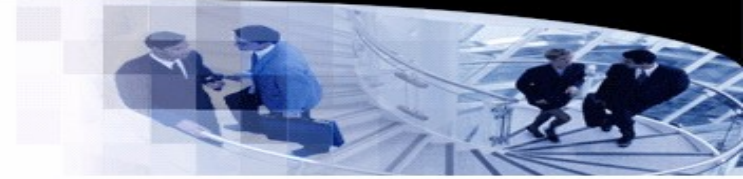
- By set operations:

- $\{a\}^+ \{1, a\}^*$

- Grammar

- Consider a languages supporting containing the following statement.
 - let $f \ x \ y = \text{if } x = 0 \text{ then } x + (\text{if } y = 0 \text{ then } 1 \text{ else } 2) \text{ else } 3$
- How can we define this language? We use grammars.

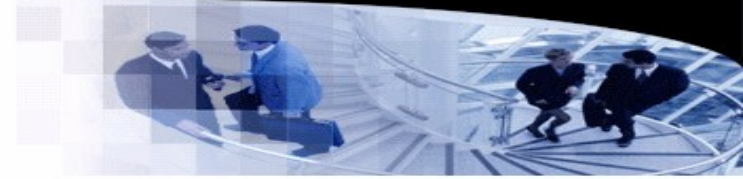
Grammar



- **Definition**

- A grammar describes which strings from an alphabet of a language is valid (well-formed) according to the syntax of the language.
- A grammar is formally defined as a tuple $G = (V_N, V_T, P, S)$, where
 - V_N is a finite set of nonterminal symbols,
 - V_T is a finite set of terminal symbol,
 - P is a set of production rules,
 - and S is the start symbol.
- A grammar tells how to **produce** (generate) all well-formed strings of its language.

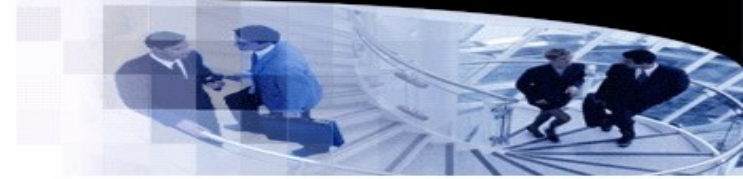
Grammar



- **Production**

- A production is an ordered pair $(\alpha, \beta) \in P$. It can be written as $\alpha \rightarrow \beta$ or $\alpha ::= \beta$.
- $\alpha \rightarrow \beta$ is a rule indicating that a substring α can be replaced by β .
- The replacement process is called **derivation**, denoted as $\gamma\alpha\delta \Rightarrow \gamma\beta\delta$.
- A sequence of derivation $\alpha = \beta_0 \Rightarrow \beta_1 \Rightarrow \dots \Rightarrow \beta_n = \gamma$ is denoted as $\alpha \xRightarrow{+} \gamma$.
- $\alpha \xRightarrow{*} \gamma$ if $\alpha \xRightarrow{+} \gamma$ or $\alpha = \gamma$.
- Productions are used to “produce” new strings from old strings.

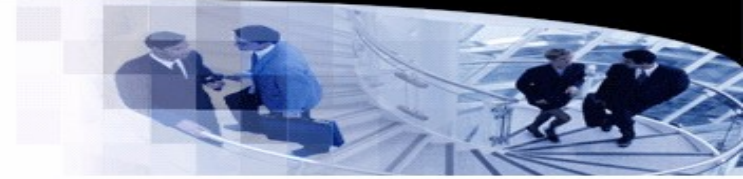
Grammar



• Example

- Let grammar $G_1 = (V_N, V_T, P, S)$, where
 - $V_N = \{S\}$,
 - $V_T = \{0, 1\}$,
 - and $P = \{(S, 1S0), (S, 10)\}$.
- G_1 can be defined in a more simple way: $G_1[S]: S \rightarrow 1S0 \mid 10$.
 - $G[X]$ indicates that X is the start symbol of the grammar.
 - $S \rightarrow 1S0 \mid 10$ is the abbreviation of two production rules (i.e., $S \rightarrow 1S0$ and $S \rightarrow 10$).
 - Conventions: Uppercase letters are nonterminal symbols. Lowercase letters and digits are terminal symbols. Lowercase Greek letters stand for strings. Uppercase letters are sets.
- We have
 - $1S0 \Rightarrow 11S00, 1S0 \Rightarrow 1100, 10S00 \Rightarrow 101000, 1S0 \overset{*}{\Rightarrow} 111S000$, and $1S0 \overset{*}{\Rightarrow} 111000$.
- If $S \overset{*}{\Rightarrow} \alpha$, what characteristics does α have?

Grammar



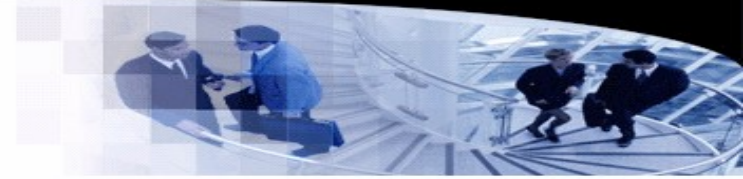
- Languages

- Given a grammar G , the language of G is defined as $\mathcal{L}(G) = \{\alpha \mid S \xRightarrow{*} \alpha \text{ and } \alpha \in \Sigma^*\}$.

- Example

- Grammar $G_2[E]$:
 - $E \rightarrow E + T \mid T$
 - $T \rightarrow T * F \mid F$
 - $F \rightarrow (E) \mid i$
- Describe $\mathcal{L}(G)$ in natural language.
- Show that $E \xRightarrow{*} i + i * i$ and $E \xRightarrow{*} (i + i) * 1$.

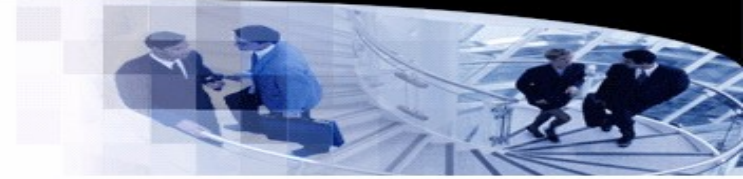
Grammar



- Chomsky hierarchy

- Type-0
 - Unrestricted grammar.
 - Example: $a \rightarrow AB$
- Type-1
 - Context-sensitive grammar. Production rules should be of the form $\beta A \gamma \rightarrow \beta \alpha \gamma$.
 - Example: $aB \rightarrow ab. \{a^n b^n c^n \mid n > 0\}$.
- Type-2
 - Context-free grammar. Production rules should be of the form $A \rightarrow \alpha$.
 - Example: $S \rightarrow aSb. \{a^n b^n \mid n > 0\}$.
- Type-3
 - Regular grammar. Production rules should be of the form $A \rightarrow aA$ or $A \rightarrow a$.
 - Example: $S \rightarrow 1S. \{1^n 0 1^m \mid m, n > 0\}$.

Grammar



- **Syntax tree**

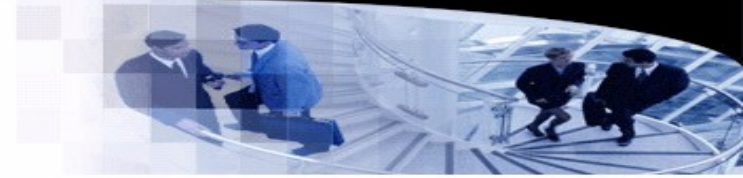
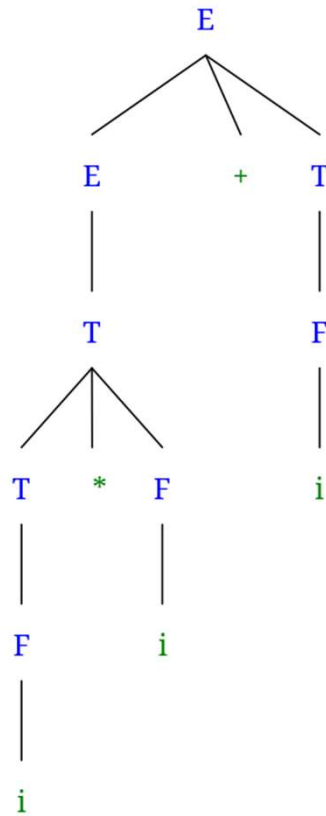
- Syntax trees (or parse tree) are used to represent structural information (syntactic structures) of sentences (well-formed strings).
- Given a context-free grammar $G = (V_N, V_T, P, S)$, a syntax tree is a tree where
 - each node is marked with a symbol $a \in V_N \cup V_T$,
 - the root node is marked with S ,
 - each non-leaf node is marked with a nonterminal symbol,
 - and, if a node is marked with A and its children are marked with $B_1, B_2, \dots, B_n$, then $A \rightarrow B_1 B_2 \dots B_n$ is a production.
- Each well-formed string has a **corresponding syntax tree**, such that the concatenation of the leaf node markers (from left to right) is the string.

Grammar

- Syntax tree

- $G_2[E]$:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow (E) \mid i$



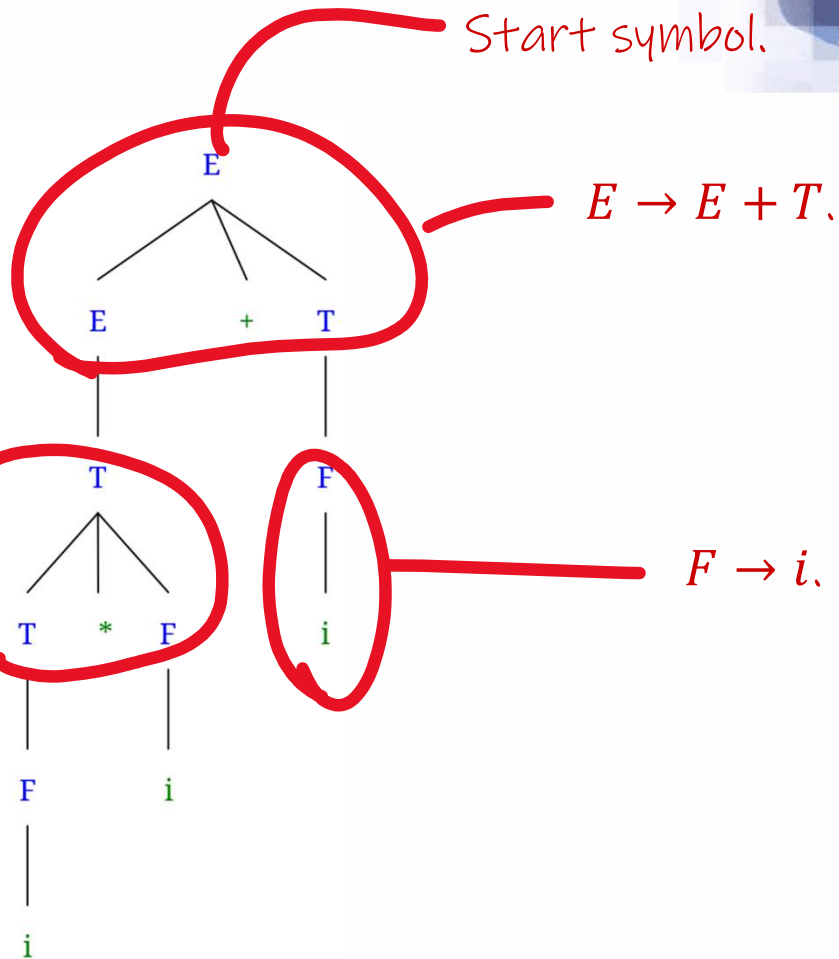
Grammar

- Syntax tree

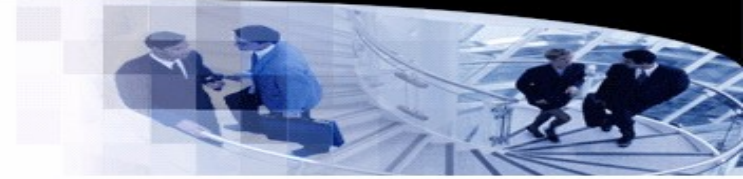
- $G_2[E]$:

- $E \rightarrow E + T \mid T$
- $T \rightarrow T * F \mid F$
- $F \rightarrow (E) \mid i$

$F \rightarrow T * F,$

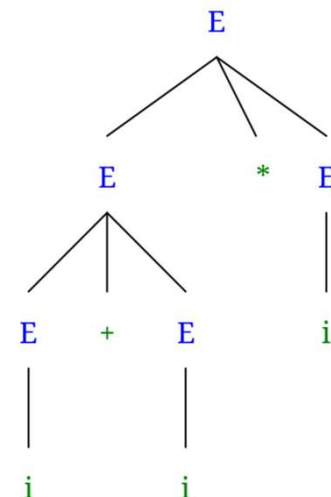
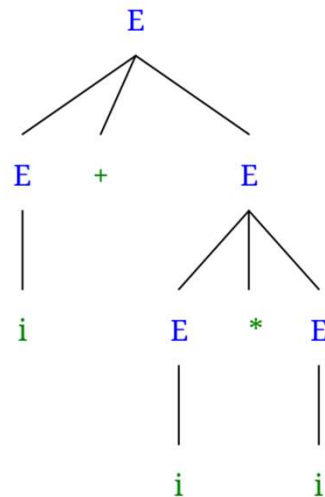


Grammar

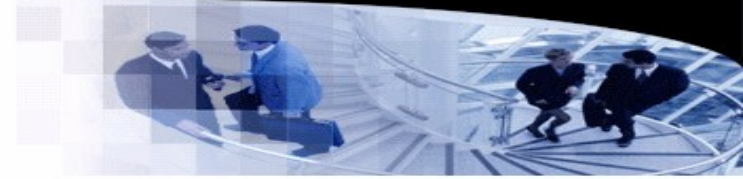


- Ambiguous grammar

- An ambiguous grammar is a context-free grammar for which there exists a string that can have more than one syntax tree.
- Given a grammar $G_3[E]: E \rightarrow E + E \mid E * E \mid (E) \mid i$, it is ambiguous because “ $i + i * i$ ” can have two syntax trees.



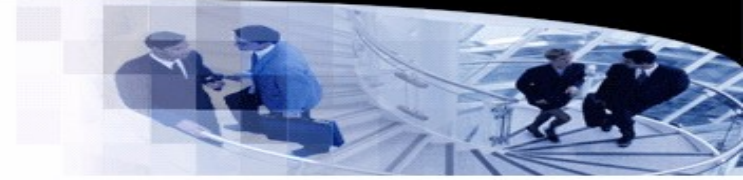
Grammar



- **Ambiguous grammar**

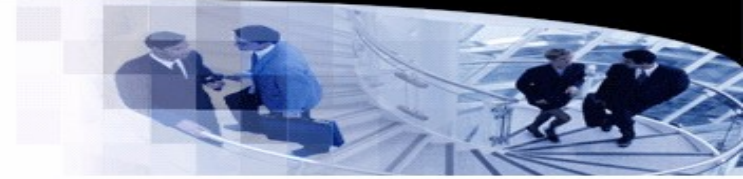
- The dangling else problem: Given $G_4[S] = \text{if } B \text{ } S \mid \text{if } B \text{ } S \text{ else } S \mid i$, it is ambiguous because “*if B if B i else i*” can have two syntax trees.
- The ambiguity can be removed by an equivalent grammar.
- $G'_4[S]$:
 - $S \rightarrow \text{matched} \mid \text{unmatched}$
 - $\text{matched} \rightarrow \text{if } B \text{ matched else matched} \mid i$
 - $\text{unmatched} \rightarrow \text{if } B \text{ } S \mid \text{if } B \text{ then matched else unmatched}$
- Show that “*if B if B i else i*” has only one syntax tree in G'_4 .

Exercises



- Given a language $L_1 = \{a, b, c\}$, write an equivalent grammar G_1 .
- $G_1[S]$:
 - $S \rightarrow a \mid b \mid c$
- Show that $\mathcal{L}(G_1) = L_1$
 - Each well-formed string of G_1 is in L_1 .
 - Each string in L_1 can be generated by G_1 .

Exercises



- Let the alphabet be $\Sigma = \{0, 1\}$. Given the following languages, write equivalent grammars.
 - $L_2 = \{ \omega \mid \omega \text{ contains at least two "1"s.} \}$
 - $L_3 = \{ \omega \mid \omega \text{ starts and end with the same symbol.} \}$
 - $L_4 = \{ \omega \mid \omega \text{ has the same number of "0"s and "1"s.} \}$
 - $L_5 = \{ \omega \mid \text{The number 1 in } \omega \text{ is more than that of 0.} \}$
 - $L_6 = \text{the complement set of } \{ 0^n 1^n \mid n \geq 0 \}.$